

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

**ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ № 4**

**«ЗАПРОСЫ НА ВЫБОРКУ И МОДИФИКАЦИЮ ДАННЫХ. ПРЕДСТАВЛЕНИЯ. РАБОТА С
ИНДЕКСАМИ»**

по дисциплине «Проектирование и реализация баз данных»

Обучающийся Остапишин Никита Алексеевич

Факультет прикладной информатики

Группа К3241

Направление подготовки 09.03.03 Прикладная информатика

Образовательная программа Мобильные и сетевые технологии

Преподаватель Говорова Марина Михайловна

Санкт-Петербург

2025/2026

Цель работы

Целью лабораторной работы является овладение практическими навыками создания запросов на выборку данных к базе данных PostgreSQL, создания представлений, использования подзапросов при модификации данных, а также создания и анализа индексов с помощью команды EXPLAIN.

Практическое задание

В ходе выполнения лабораторной работы необходимо:

1. Создать запросы на выборку данных к базе данных PostgreSQL согласно индивидуальному заданию.
2. Создать представления на основе запросов.
3. Составить три запроса на модификацию данных: INSERT, UPDATE, DELETE с использованием подзапросов.
4. Изучить графическое представление запросов и историю запросов в pgAdmin 4.
5. Создать простой и составной индексы для двух запросов.
6. Сравнить время выполнения запросов без индексов и с индексами с помощью EXPLAIN ANALYZE.
7. Удалить созданные индексы.

Схема базы данных

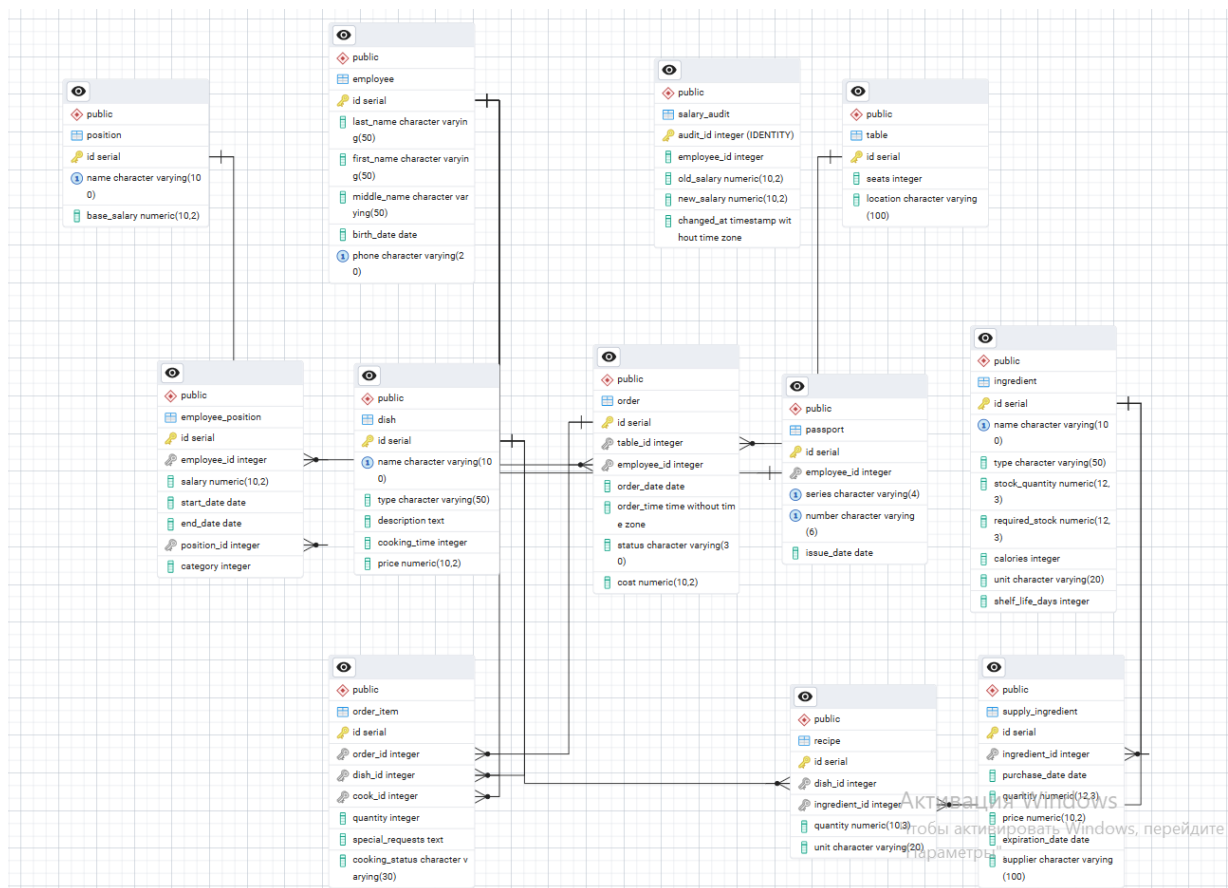


Рисунок 1 – ERD-схема

Выполнение работы

1. Создание запросов к базе данных

Запросы выполнялись в инструменте Query Tool среды pgAdmin 4. Для каждого запроса была введена SQL-команда, после чего результат выполнения отображался на вкладке Data Output.

Запрос 1. Вывести данные официанта, принявшего заказы на максимальную сумму за истекший месяц.

Query

Query History

1

2

3

4

5

6

7

8

SELECT e.first_name, e.last_name, SUM(o.cost) AS total_sum

FROM employee e

JOIN "order" o ON e.id = o.employee_id

WHERE o.order_date >= CURRENT_DATE - INTERVAL '1 month'

GROUP BY e.id, e.first_name, e.last_name

ORDER BY total_sum DESC LIMIT 1;

Data Output

Messages

Notifications

≡+

📄

▼

📋

▼

🗑️

🗄️

⬇️

📈

SQL

Show

	first_name character varying (50) 🔒	last_name character varying (50) 🔒	total_sum numeric 🔒
1	Екатерина	Петрова	6550.00

an

Total rows: 1

Query complete 00:00:00.183

Рисунок 2 – Результат выполнения запроса 1

Запрос 2

Рассчитать премию каждого официанта за последние 10 дней (5% от стоимости каждого заказа).

The screenshot shows a SQL query editor with a query window and a data output window. The query is as follows:

```
1 SELECT e.first_name, e.last_name, SUM(o.cost * 0.05) AS bonus_5pct
2 FROM employee e
3 JOIN "order" o ON e.id = o.employee_id
4 WHERE o.order_date >= CURRENT_DATE - INTERVAL '10 days'
5 GROUP BY e.id, e.first_name, e.last_name;
6
```

The data output window shows the results of the query:

	first_name character varying (50)	last_name character varying (50)	bonus_5pct numeric
1	Мария	Кузнецова	86.0000
2	Екатерина	Петрова	167.5000

At the bottom, the status bar indicates: Total rows: 2 | Query complete 00:00:00.117

Рисунок 3 – Результат запроса 2

Запрос 3

Подсчитать, сколько ингредиентов содержит каждое блюдо.

Query

Query History

```

1 SELECT d.name AS dish_name, COUNT(r.ingredient_id) AS ingr_count
2 FROM dish d
3 LEFT JOIN recipe r ON d.id = r.dish_id
4 GROUP BY d.id, d.name
5 ORDER BY ingr_count DESC;
6

```

Data Output

Messages

Notifications

+

📄

▼

🗑️

📄

📄

📄

📄

SQL

	dish_name character varying (100) 🔒	ingr_count bigint 🔒
1	Паста Карбонара	3
2	Греческий салат	3
3	Цезарь с курицей	3
4	Суп-пюре грибной	2
5	Стейк Рибай	1
6	Тирамису	1

Total rows: 6

Query complete 00:00:00.119

Запрос 4

Вывести название блюда, содержащее максимальное число ингредиентов.

Query

Query History

1

SELECT d.name, COUNT(r.ingredient_id) AS ingr_count

2

FROM dish d

3

JOIN recipe r ON d.id = r.dish_id

4

GROUP BY d.id, d.name

5

ORDER BY ingr_count DESC LIMIT 1;

6

Data Output

Messages

Notifications

SQL

	name character varying (100)	ingr_count bigint
1	Паста Карбонара	3

Total rows: 1

Query complete 00:00:00.124

Запрос 5

Какой повар может приготовить максимальное число видов блюд?

Query Query History

```
1 SELECT e.first_name, e.last_name, COUNT(DISTINCT oi.dish_id) AS unique_dishes
2 FROM employee e
3 JOIN order_item oi ON e.id = oi.cook_id
4 GROUP BY e.id, e.first_name, e.last_name
5 ORDER BY unique_dishes DESC LIMIT 1;
```

Data Output Messages Notifications

SQL

first_name

character varying (50)

last_name

character varying (50)

unique_dishes

bigint

1

Алексей

Сидоров

4

Total rows: 1 Query complete 00:00:00.131

Рисунок 6 – Результат запроса 5

Запрос 6

Сколько закреплено столов за каждым из официантов за сегодняшний

Query

Query History

1

```
SELECT e.first_name, e.last_name, COUNT(DISTINCT o.table_id) AS tables_today
FROM employee e
JOIN "order" o ON e.id = o.employee_id
WHERE o.order_date = CURRENT_DATE
GROUP BY e.id, e.first_name, e.last_name;
```

Data Output

Messages

Notifications

≡

📄

▼

📋

▼

🗑️

📦

⬇️

📈

SQL

Sho

	first_name character varying (50) 🔒	last_name character varying (50) 🔒	tables_today bigint 🔒
1	Екатерина	Петрова	1
2	Мария	Кузнецова	1

день?

Total rows: 2 Query complete 00:00:00.093

Рисунок 7 – Результат запроса 6

Запрос 7

Какой из ингредиентов используется в максимальном количестве блюд?

The screenshot shows a database query interface. At the top, there are tabs for 'Query' and 'Query History'. The 'Query' tab is active, displaying the following SQL query:

```
1 SELECT i.name, COUNT(DISTINCT r.dish_id) AS dishes_count
2 FROM ingredient i
3 JOIN recipe r ON i.id = r.ingredient_id
4 GROUP BY i.id, i.name
5 ORDER BY dishes_count DESC LIMIT 1;
```

Below the query editor, there are tabs for 'Data Output', 'Messages', and 'Notifications'. The 'Data Output' tab is active, showing a table with the results of the query. The table has two columns: 'name' (character varying (100)) and 'dishes_count' (bigint). The first row shows 'Сливки 33%' with a count of 3.

	name character varying (100)	dishes_count bigint
1	Сливки 33%	3

At the bottom of the interface, there is a status bar showing 'Total rows: 1' and 'Query complete 00:00:00.081'.

Рисунок 8 – Запрос результат 7

Рисунок —

2. Создание представлений

Представления создавались с помощью команды CREATE VIEW. После создания каждого представления выполнялся запрос SELECT * FROM view_name, чтобы проверить его содержимое

Представление 1

стоимость ингредиентов для заданного блюда

Query

Query History

1

CREATE VIEW v_dish_ingredient_cost AS

2

SELECT

3

d.name AS dish_name, i.name AS ingredient_name,

4

r.quantity, r.unit, s.price, (r.quantity * s.price) AS ingredient_cost

5

FROM dish d

6

JOIN recipe r ON d.id = r.dish_id

7

JOIN ingredient i ON r.ingredient_id = i.id

8

LEFT JOIN supply_ingredient s ON i.id = s.ingredient_id;

Data Output

Messages

Notifications

CREATE VIEW

Query returned successfully in 133 msec.

Total rows:

Query complete 00:00:00.133

Рисунок —

Query

Query History

1

SELECT * FROM v_dish_ingredient_cost WHERE dish_name = 'Цезарь с курицей';

Data Output

Messages

Notifications

+

📄

▼

📋

▼

🗑️

📚

⬇️

📈

SQL

Showing rows: 1 to 3

✎

	dish_name character varying (100)	ingredient_name character varying (100)	quantity numeric (10,3)	unit character varying (20)	price numeric (10,2)	ingredient_cost numeric
1	Цезарь с курицей	Куриное филе	0.150	кг	350.00	52.50000
2	Цезарь с курицей	Салат Айсберг	0.100	кг	120.00	12.00000
3	Цезарь с курицей	Пармезан	0.030	кг	1200.00	36.00000

Активация

Чтобы активировать

Параметры

Successfully run. Total

Total rows: 3

Query complete 00:00:00.119

Рисунок –
Представление 2
количество приготовленных блюд поваром за дату

```
Query    Query History
1  CREATE VIEW v_cook_stats_by_date AS
2  SELECT
3      e.first_name || ' ' || e.last_name AS cook_name,
4      d.name AS dish_name, o.order_date, COUNT(oi.id) AS prepared_qty
5  FROM order_item oi
6  JOIN "order" o ON oi.order_id = o.id
7  JOIN employee e ON oi.cook_id = e.id
8  JOIN dish d ON oi.dish_id = d.id
9  GROUP BY e.first_name, e.last_name, d.name, o.order_date;
```

Data Output	Messages	Notifications
CREATE VIEW	Query returned successfully in 113 msec.	
Total rows:	Query complete 00:00:00.113	

Query Query History

```
1 SELECT * FROM v_cook_stats_by_date WHERE order_date = CURRENT_DATE;
```

Total rows: 4 Query complete 00:00:00.080

Рисунок —

3. Запросы на модификацию данных с использованием подзапросов

Для выполнения данного пункта были составлены запросы INSERT, UPDATE и DELETE, в которых используются подзапросы. Перед и после выполнения каждого запроса выполнялся SELECT, чтобы сравнить состояние данных.

3.1. Update с использованием подзапроса

Формулировка запроса:

Повысить зарплату на 5% сотрудникам, которые приняли 2 и более заказов.

Query

Query History

1

SELECT ep.id, e.last_name, e.first_name, ep.salary, ep.start_date

2

FROM employee_position ep

3

JOIN employee e ON ep.employee_id = e.id

4

WHERE ep.end_date IS NULL

5

ORDER BY e.last_name;

Data Output

Messages

Notifications

+

📄

▼

🗑️

🗑️

👤

📶

SQL

	id integer 🔒	last_name character varying (50) 🔒	first_name character varying (50) 🔒	salary numeric (10,2) 🔒	start_date date 🔒
1	5	Волков	Дмитрий	55000.00	2021-07-10
2	1	Иванов	Иван	60000.00	2020-01-10
3	4	Кузнецова	Мария	40000.00	2022-02-01
4	2	Петрова	Екатерина	40000.00	2021-03-15
5	3	Сидоров	Алексей	55000.00	2019-11-20

Total rows: 5

Query complete 00:00:00.080

Рисунок —

Query

Query History

1

UPDATE employee_position

2

SET salary = salary * 1.05

3

WHERE end_date IS NULL

4

AND employee_id IN (

5

SELECT employee_id

6

FROM "order"

7

GROUP BY employee_id

8

HAVING COUNT(id) >= 2

9

);

Data Output

Messages

Notifications

ЗАМЕЧАНИЕ: Аудит: сотрудник id=4 — оклад изменён с 40000.00 на 42000.00 руб.

ЗАМЕЧАНИЕ: Аудит: сотрудник id=2 — оклад изменён с 40000.00 на 42000.00 руб.

UPDATE 2

Query returned successfully in 91 msec.

Query

Query History

1

SELECT ep.id, e.last_name, e.first_name, ep.salary, ep.start_date

2

FROM employee_position ep

3

JOIN employee e ON ep.employee_id = e.id

4

WHERE ep.end_date IS NULL

5

ORDER BY e.last_name;

Data Output

Messages

Notifications

≡+

📄

▼

📄

🗑️

📦

⬇️

📈

SQL

	id integer 🔒	last_name character varying (50) 🔒	first_name character varying (50) 🔒	salary numeric (10,2) 🔒	start_date date 🔒
1	5	Волков	Дмитрий	55000.00	2021-07-10
2	1	Иванов	Иван	60000.00	2020-01-10
3	4	Кузнецова	Мария	42000.00	2022-02-01
4	2	Петрова	Екатерина	42000.00	2021-03-15
5	3	Сидоров	Алексей	55000.00	2019-11-20

Рисунок –

3.2. Insert с использованием подзапроса

Формулировка запроса: Добавить в активный заказ №2 новое блюдо "Тирамису", автоматически определив его ID и ID повара "Сидоров" через подзапросы.

Рисунок –

QueryQuery History

12345

```
SELECT oi.id, d.name AS dish, e.last_name AS cook, oi.quantity, oi.cooking_status
FROM order_item oi
JOIN dish d ON oi.dish_id = d.id
LEFT JOIN employee e ON oi.cook_id = e.id
WHERE oi.order_id = 2;
```

Data OutputMessagesNotifications

Showing rows

	id integer	dish character varying (100)	cook character varying (50)	quantity integer	cooking_status character varying (30)
1	3	Суп-пюре грибной	Сидоров	1	cooking
2	4	Тирамису	Волков	2	pending

✓ Success

Total rows: 2Query complete 00:00:00.086

Рисунок –

Query	Query History
1	INSERT INTO order_item (order_id, dish_id, cook_id, quantity, cooking_status)
2	VALUES (
3	2,
4	(SELECT id FROM dish WHERE name = 'Тирамису'),
5	(SELECT id FROM employee WHERE last_name = 'Сидоров' AND first_name = 'Алексей'),
6	1,
7	'pending'
8	);

Data Output	Messages	Notifications
INSERT 0 1		
Query returned successfully in 65 msec.		

Ак
Что
"Па

Total rows:	Query complete 00:00:00.065
-------------	-----------------------------

Рисунок —

Query

Query History

```

1  SELECT oi.id, d.name AS dish, e.last_name AS cook, oi.quantity, oi.cooking_status
2  FROM order_item oi
3  JOIN dish d ON oi.dish_id = d.id
4  LEFT JOIN employee e ON oi.cook_id = e.id
5  WHERE oi.order_id = 2;

```

Data Output

Messages

Notifications

≡

📄

▼

📋

▼

🗑️

🗄️

⬇️

📈

SQL

Showing rows

	id integer	dish character varying (100)	cook character varying (50)	quantity integer	cooking_status character varying (30)
1	3	Суп-пюре грибной	Сидоров	1	cooking
2	4	Тирамису	Волков	2	pending
3	10	Тирамису	Сидоров	1	pending

Total rows: 3

Query complete 00:00:00.059

✓

Success

Рисунок —

3.3. DELETE с использованием подзапроса

Удалить позиции из заказов, которые находятся в статусе 'pending' и относятся к блюдам типа 'Десерт'.

Рисунок –

Query

Query History

1

`SELECT oi.id, oi.order_id, d.name AS dish, d.type, oi.cooking_status, oi.quantity`

2

`FROM order_item oi`

3

`JOIN dish d ON oi.dish_id = d.id`

4

`WHERE oi.cooking_status = 'pending';`

Data Output

Messages

Notifications

SQL

Showing rows:

	id integer	order_id integer	dish character varying (100)	type character varying (50)	cooking_status character varying (30)	quantity integer
1	10	2	Тирамису	Десерт	pending	1
2	4	2	Тирамису	Десерт	pending	2
3	6	4	Греческий салат	Салат	pending	1

✓ Success

Total rows: 3 Query complete 00:00:00.094

Рисунок –

Query	Query History
1	DELETE FROM order_item
2	WHERE cooking_status = 'pending'
3	AND dish_id IN (
4	SELECT id FROM dish WHERE type = 'Десерт'
5	);

Data Output	Messages	Notifications
DELETE 2		
Query returned successfully in 80 msec.		

Total rows:	Query complete 00:00:00.080
-------------	-----------------------------

Рисунок –

Query

Query History

1

SELECT oi.id, oi.order_id, d.name AS dish, d.type, oi.cooking_status, oi.quantity

2

FROM order_item oi

3

JOIN dish d ON oi.dish_id = d.id

4

WHERE oi.cooking_status = 'pending';

Data Output

Messages

Notifications

≡+

▼

▼

SQL

Showing 1

	id integer	order_id integer	dish character varying (100)	type character varying (50)	cooking_status character varying (30)	quantity integer
1	6	4	Греческий салат	Салат	pending	1

✓ Su

Total rows: 1

Query complete 00:00:00.092

4. Изучение графического представления запросов и истории запросов

В pgAdmin 4 был использован инструмент Query Tool. На верхней панели инструмента вводились SQL-команды. После выполнения запросов результаты отображались на вкладке Data Output, а сообщения сервера — на вкладке Messages.

Для просмотра плана выполнения запроса использовалась вкладка Explain. С помощью команды EXPLAIN и режима EXPLAIN ANALYZE был получен план выполнения запроса. В графическом представлении отображались операции, которые PostgreSQL использует для выполнения запроса, например Seq Scan, Index Scan, Hash Join, Nested Loop.

Также была просмотрена вкладка Query History, где отображаются ранее выполненные SQL-команды, время выполнения, количество возвращенных строк и сообщения сервера.

5. Работа с индексами

Для анализа индексов были выбраны два запроса. Сначала запросы выполнялись без дополнительных индексов, затем создавались индексы, после чего запросы выполнялись повторно. Для анализа использовалась команда EXPLAIN ANALYZE.

5.1. Простой индекс

Рисунок –

QueryQuery History

1

EXPLAIN ANALYZE SELECT id, table_id, employee_id, order_date, cost

2

FROM "order" WHERE status = 'completed';

Data OutputMessagesNotifications

SQL

Sh

	QUERY PLAN
1	Seq Scan on "order" (cost=0.00..16.88 rows=3 width=32) (actual time=0.036..0.038 rows=2 loops=...
2	Filter: ((status)::text = 'completed'::text)
3	Rows Removed by Filter: 3
4	Planning Time: 0.142 ms
5	Execution Time: 0.072 ms

CREATE INDEX idx_order_status ON "order"(status);

Рисунок –

Query

Query History

1

EXPLAIN ANALYZE SELECT id, table_id, employee_id, order_date, cost

2

FROM "order" WHERE status = 'completed';

Data Output

Messages

Notifications

≡+

▼

▼

SQL

QUERY PLAN

text

1

Seq Scan on "order" (cost=0.00..1.06 rows=1 width=32) (actual time=0.035..0.038 rows=2 loops=...

2

Filter: ((status)::text = 'completed'::text)

3

Rows Removed by Filter: 3

4

Planning Time: 1.425 ms

5

Execution Time: 0.057 ms

Total rows: 5

Query complete 00:00:00.071

5.2. Составной индекс

Формулировка запроса: найти договоры по мастерской и дате заключения.

Рисунок – Без индексов

Query Query History

```
1 EXPLAIN ANALYZE
2 SELECT e.last_name, e.first_name, SUM(o.cost)
3 FROM "order" o
4 JOIN employee e ON o.employee_id = e.id
5 WHERE o.order_date = CURRENT_DATE
6 GROUP BY e.id, e.last_name, e.first_name;
```

Data Output Messages Notifications

Showing rows: 1 to 13

	QUERY PLAN
1	GroupAggregate (cost=9.34..9.36 rows=1 width=272) (actual time=0.037..0.040 rows=2 loops=1)
2	Group Key: e.id
3	-> Sort (cost=9.34..9.34 rows=1 width=256) (actual time=0.028..0.029 rows=2 loops=1)
4	Sort Key: e.id
5	Sort Method: quicksort Memory: 25kB
6	-> Nested Loop (cost=0.14..9.33 rows=1 width=256) (actual time=0.019..0.022 rows=2 loops=1)
7	-> Seq Scan on "order" o (cost=0.00..1.07 rows=1 width=20) (actual time=0.011..0.013 rows=2 loops=1)
8	Filter: (order_date = CURRENT_DATE)
9	Rows Removed by Filter: 3
10	-> Index Scan using employee_pkey on employee e (cost=0.14..8.16 rows=1 width=240) (actual time=0.003..0.003 rows=...
11	Index Cond: (id = o.employee_id)
12	Planning Time: 0.588 ms
13	Execution Time: 0.102 ms

Total rows: 13 Query complete 00:00:00.102

Рисунок – Прописываю

CREATE INDEX idx_order_date_employee ON "order"(order_date, employee_id);

Рисунок – С индексами

Query

Query History

1

EXPLAIN ANALYZE

2

SELECT e.last_name, e.first_name, SUM(o.cost)

3

FROM "order" o

4

JOIN employee e ON o.employee_id = e.id

5

WHERE o.order_date = CURRENT_DATE

6

GROUP BY e.id, e.last_name, e.first_name;

Data Output

Messages

Notifications

≡+

📄

▼

📋

▼

🗑️

🗑️

📥

📥

📈

SQL

Showing rows: 1

QUERY PLAN

text

🔒

1

GroupAggregate (cost=9.34..9.36 rows=1 width=272) (actual time=0.056..0.057 rows=2 loops=1)

2

Group Key: e.id

3

-> Sort (cost=9.34..9.34 rows=1 width=256) (actual time=0.036..0.037 rows=2 loops=1)

4

Sort Key: e.id

5

Sort Method: quicksort Memory: 25kB

6

-> Nested Loop (cost=0.14..9.33 rows=1 width=256) (actual time=0.025..0.029 rows=2 loops=1)

7

-> Seq Scan on "order" o (cost=0.00..1.07 rows=1 width=20) (actual time=0.015..0.017 rows=2 loops=1)

8

Filter: (order_date = CURRENT_DATE)

9

Rows Removed by Filter: 3

10

-> Index Scan using employee_pkey on employee e (cost=0.14..8.16 rows=1 width=240) (actual time=0.005..0.005 rows=...

11

Index Cond: (id = o.employee_id)

12

Planning Time: 2.324 ms

13

Execution Time: 0.097 ms

Total rows: 13

Query complete 00:00:00.135

6. Выводы

В ходе выполнения лабораторной работы были получены практические навыки работы с запросами на выборку данных в PostgreSQL. Были составлены SQL-запросы с использованием соединений таблиц, группировки, агрегатных функций и общих табличных выражений.

Также были созданы представления, которые позволяют упростить дальнейшую работу с часто используемыми запросами. Представления были проверены с помощью команды SELECT.

Были выполнены запросы на модификацию данных INSERT, UPDATE и DELETE с использованием подзапросов. Перед и после выполнения запросов выполнялась проверка данных, что позволило оценить изменения в таблицах.

Кроме того, были изучены планы выполнения запросов с помощью команды EXPLAIN ANALYZE и графического представления во вкладке Explain в pgAdmin 4. Для двух запросов были созданы простой и составной индексы. После сравнения времени выполнения запросов было установлено, что индексы могут ускорить поиск данных по часто используемым условиям отбора. После завершения анализа созданные индексы были удалены.